

Conceptos de Go aplicados en el proyecto de Quiz

Este documento resume los principales conceptos y herramientas del lenguaje Go utilizados para construir el proyecto del juego de preguntas y respuestas (quiz). Está pensado como una referencia rápida para repasar las ideas prácticas aplicadas en el código.

1. Lectura de archivos y parsing CSV

Se utilizó el paquete `encoding/csv` para leer archivos `.csv` con preguntas y respuestas. - Se abrió el archivo con `os.Open()`. - Se leyó con `csv.NewReader(file).ReadAll()`.

Esto permite importar quizzes desde cualquier archivo externo.

2. Flags de entrada

Con el paquete `flag`, se implementaron opciones para personalizar el comportamiento del programa desde la línea de comandos:

- `-csv` permite indicar el archivo de preguntas.
- `-limit` define el tiempo límite del quiz (en segundos).
- `-shuffle` permite mezclar el orden de las preguntas.

Esto facilita reusabilidad y configurabilidad sin cambiar el código.

3. Estructuras de control y slices

- Se recorren todas las preguntas usando `for _, record := range records`.
- Se cuentan las respuestas correctas con un entero (`correctas++`).

Se utilizan slices para representar las filas del CSV y acceder a `record[0]` (pregunta) y `record[1]` (respuesta esperada).

4. Normalización de entradas del usuario

Para evitar que errores de espacio o capitalización generen respuestas falsas:

- Se usó `strings.TrimSpace()` para eliminar espacios.
- Se aplicó `strings.ToLower()` para comparar todo en minúsculas.

Esto mejora la experiencia del usuario.

5. Concurrency: goroutines y canales

El quiz incorpora **concurrency** para poder controlar el tiempo sin bloquear la entrada del usuario:

- Se lanza una `goroutine` para leer la respuesta del usuario con `fmt.Scanln()`.
- Se usa un `chan string` para comunicar esa respuesta.
- Un `select` permite esperar o por la respuesta del usuario o por el fin del tiempo con un `timer`.

Esto hace posible cortar el juego incluso si el usuario está pensando y no ha respondido.

6. Temporizadores

Con el paquete `time`:

- Se creó un timer con `time.NewTimer(...)` usando el flag `-limit`.
- Se escuchó el canal `timer.C` para saber cuándo se agotó el tiempo.

Permite limitar el quiz a un tiempo real de ejecución.

7. Mezclar preguntas aleatoriamente

Con el paquete `math/rand`:

- Se activó el modo aleatorio con `rand.Seed(time.Now().UnixNano())`.
- Se mezcló el slice `records` usando `rand.Shuffle(...)`.

Esto hace que el orden de las preguntas cambie en cada ejecución (si el flag `-shuffle` está activado).

8. Salida de resultados

El programa imprime el número de respuestas correctas sobre el total:

```
fmt.Printf("Respuestas correctas: %d de %d\n", correctas, len(records))
```

Esto ocurre tanto si se termina el quiz por tiempo como si el usuario responde todas las preguntas.

9. Control de flujo y retorno anticipado

Cuando el tiempo se agota, se imprime el resultado parcial y se ejecuta `return` para terminar el `main()` inmediatamente:

```
case <-timer.C:  
    fmt.Println("\n🕒 ¡Tiempo terminado!")  
    fmt.Printf("Respuestas correctas: %d de %d\n", correctas, len(records))  
    return
```

Esto evita seguir preguntando al usuario innecesariamente.

Conclusión

Este ejercicio permite practicar una gran cantidad de conceptos de Go de forma integrada: - E/S de archivos - Flags de terminal - Estructuras de datos - Concurrencia - Control de flujo - Temporizadores

Es un buen ejemplo pequeño pero realista de una aplicación CLI concurrente con entrada del usuario.